

Correcting topological errors using SSNbler and ArcGIS Pro

David Nagel, Erin E Peterson, Alan Pearse

October 27, 2025

Contents

1. Introduction	1
2. Example data and software requirements	1
3. The Landscape Network.....	2
3.1 Valid node categories	2
3.2 Types of topology errors	3
3.3 Additional topological restrictions	4
4. Identifying topological errors and restrictions in R.....	5
5. Verifying and correcting topological errors in ArcGIS Pro.....	7
5.2 Check the nodecat values	9
5.3 Edit various topology errors	11

1. Introduction

To generate a valid landscape network (LSN) with the SSNbler R package, users must ensure their digitised streams are free of topological errors and topological restrictions not permitted in SSN2. Vector streams data are sometimes prone to topological errors, even from validated public domain sources. The aim of this tutorial is to illustrate the common topological errors sometimes found in digital stream networks and provide guidance on correcting those. It has been written for SSNbler v1.1.0 and assumes the user has some familiarity with ArcGIS Pro ver. 3.0 or greater, and a good working knowledge of R Statistical Software version $\geq 4.0.0$ (R Core Team 2024) software.

2. Example data and software requirements

This tutorial comes with an example dataset and R script. The dataset can be found in the zip archive, **topology_pro.zip**. The topology dataset should contain “ARiver.shp” (and associated files), which is a shapefile of streams data containing two sub-networks. It should also contain the subfolder, “final_outputs,” which contains a reference version of the final error-free LSN created in this tutorial,

along with an empty folder called `\work` where working results will be stored. The R script, `CheckTopologyPro.R`, contains the R code described in this tutorial.

To begin this tutorial, first navigate to the folder **C:\temp** on your computer, or create the folder on your C: drive. Delete all the files from this folder if any exist. Copy **topology_pro.zip** to the **C:\temp** folder and unzip it. It's OK to use another folder, however path names in the R script will need to be modified to reflect the new folder name.

For this tutorial, it is assumed that the topology dataset has been unzipped and stored in the folder **C:/temp/topology_pro/**, and that a new folder called **C:/temp/topology_pro/work** has been created for writing intermediate outputs. **C:/temp/topology_pro/documents** contains this tutorial document. The folder structure on the user's computer should look something like this:

Name	Date modified	Type
documents	4/25/2024 12:35 PM	File folder
final_output	4/25/2024 12:34 PM	File folder
work	4/17/2024 11:37 AM	File folder
ARiver_Pro.CPG	4/25/2024 8:57 AM	CPG File
ARiver_Pro.dbf	4/25/2024 8:57 AM	DBF File
ARiver_Pro.prj	11/28/2023 1:43 PM	PRJ File
ARiver_Pro.sbn	4/25/2024 8:57 AM	SBN File
ARiver_Pro.sbx	4/25/2024 8:57 AM	SBX File
ARiver_Pro.shp	4/25/2024 8:57 AM	SHP File
ARiver_Pro.shp.xml	4/25/2024 9:43 AM	Microsoft Edge HTML D...
ARiver_Pro.shx	4/25/2024 8:57 AM	SHX File
CheckTopologyPro.R	4/22/2024 1:08 PM	R File

3. The Landscape Network

It is important to have a basic understanding of the structure of a Landscape Network (LSN) and the relationship between the edges and nodes within it. Streams data are represented as a collection of directed edges, where the directionality is determined by the digitized direction of the line features. A nodes dataset (nodes.gpkg) is created when the LSN is generated in R using the `lines_to_lsn` function in `SSNbler`. Each edge is associated with two nodes, which correspond to the upstream and downstream endpoints of the edge. When more than one edge flows into or out of the node, they *share* a node. Thus, there should always be a single node at the intersection of edges (Fig. 1). If there is more or less than one node at an intersection, it is a topological error. If these errors are not corrected, the connectivity between line features and the observed and prediction sites associated with them will not be accurately represented in the SSN object or the spatial statistical models subsequently fit to the data. Correcting these errors is often the most time-consuming step in the pre-processing and modelling workflow.

3.1 Valid node categories

There are four topologically valid node categories in a LSN:

- **Source:** Water flow originates at these nodes and flows downstream. An edge that begins at a source node is called a headwater segment. Source nodes do not receive flow from any upstream edges.
- **Outlet:** Flow terminates at these nodes. This node is the most downstream point in the edge network. We refer to an edge that terminates at an outlet as an outlet segment. Note that a streams dataset may contain multiple subnetworks with associated outlets.
- **Confluence:** Multiple edges converge, or flow into, a single node.
- **Pseudonode:** One edge flows in and one edge flows out of a single node. While these nodes are not topologically problematic, an excessive number of pseudonodes can slow down geoprocessing operations for large datasets.

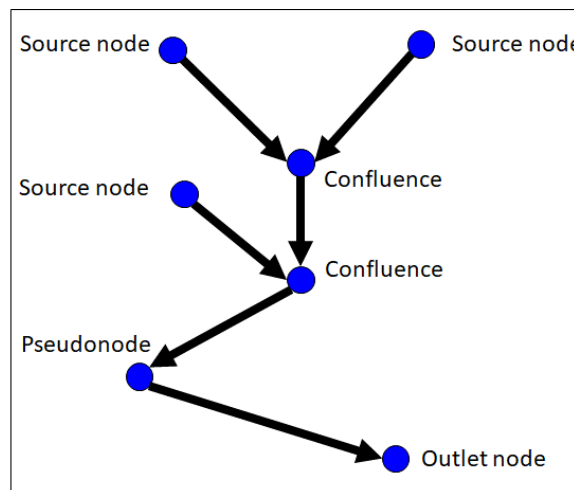


Fig. 1. An illustration of a landscape network (LSN). Nodes are denoted by blue circles, with the node category labelled. Edges are denoted by black arrows, with the arrow indicating flow direction (i.e. digitized direction).

3.2 Types of topology errors

When topological errors exist in an LSN, the flow connectivity is misrepresented, which can negatively affect statistical model outputs. Therefore, these errors must be corrected prior to fitting statistical models. Three common topological errors are recognised (Fig. 2):

1. **Unsnapped node:** occurs when two edges should be connected by a common node but are instead separated by a small distance (Fig. 2a).
2. **Dangling node:** occurs when the end node of an edge is close to another edge that it should be connected to but does not touch (Fig. 2b).
3. **Intersection without nodes:** two edges in a stream network should never intersect without a node present. This error occurs when one edge feature crosses another without a node at the intersection (Fig. 2c) or the end node of one edge feature touches another edge at any point other than the end nodes (Fig. 2d). Note that the 'outlet node' at the end of the offending line segment may also be considered a dangling node (Fig. 2c, yellow dot).

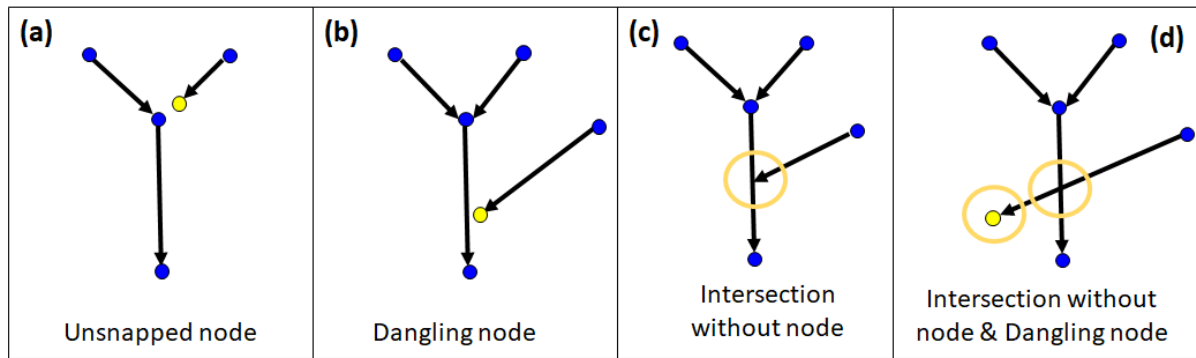


Fig. 2. Illustrations of topological errors in a stream network. The nodes represented as blue dots, with the node of interest shown as yellow dots and circles. The edges are represented using black arrows pointing in the direction of flow. Three types of topological errors are shown: a) Unsnapped node, b) Dangling node, c) Intersection without a node, and d) Intersection without node and Dangling node.

3.3 Additional topological restrictions

In addition to true topological errors, some additional topological restrictions are imposed on the nodes (Fig. 3). These include:

- **Converging node:** occurs when two edges flow into a node and no edges flow out (Fig. 3a).
- **Complex confluence:** occurs when more than two edges flow into a node (Fig. 3b). No more than two edges can flow into a confluence. If this condition is violated, the node is called a *complex confluence*.
- **Downstream divergence:** occurs when more than one edge flows out of a node (Fig. 3c). This topological condition occurs naturally in braided streams (Fig. 3d), but is not allowed in a LSN.

Although these topological conditions appear naturally in stream networks, they complicate the derivation of flow-based dependence structures in LSNs. Therefore, these cases must be adjusted or corrected.

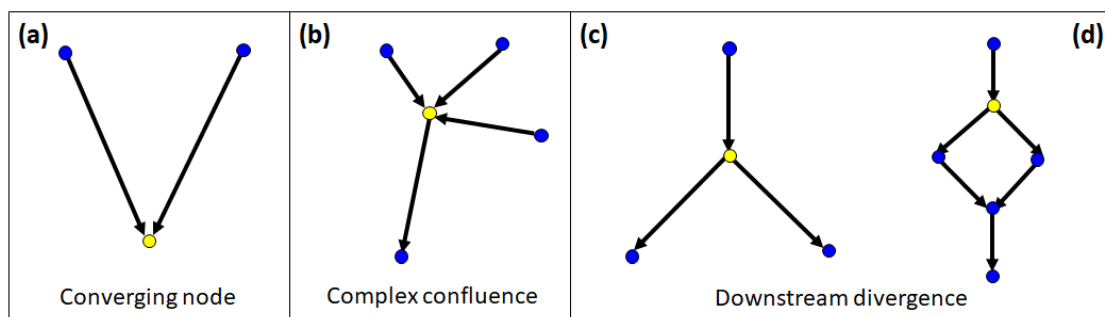


Fig. 3. Illustrations of topological features in stream networks that are restricted in landscape stream networks. Nodes are drawn as circles, and the edges are drawn as arrows pointing in the direction of flow. In each panel, the problem node is highlighted in yellow; the associated label identifies the topological restriction that is violated by the node.

The purpose of this tutorial is to demonstrate a geoprocessing workflow that may help users eliminate topological errors and restricted conditions from their stream network data using ArcGIS

Pro. In general, error-correction is an iterative process that may involve repetition of manual and semi-automatic steps in various combinations. Only manual steps are showcased in this tutorial.

4. Identifying topological errors and restrictions in R

Topological errors (Fig. 2) and restrictions (Fig. 3) can be identified using the `lines_to_lsn` function in `SSNbler`. This function does three main things:

1. Builds the LSN;
2. Checks network topology and fixes minor topological errors (optional); and
3. Writes output files to a local directory.

We will begin by walking through these steps.

Open `CheckTopologyPro.R` in RStudio or any other IDE for R. Install and load libraries for the relevant R packages. Set the number of cores for your computer for parallel processing. Set your working folder, which may be different to the one below. Run the R code step by step following the guidance below.

```
## Install packages from CRAN repository if necessary
install.packages("sf")
install.packages("SSNbler")

## Load SSNbler and other useful packages
library(SSNbler)
library(sf)
## Load two more libraries for parallel processing
library(tidyverse)
library(parallel)

## Set number of cores to use for parallel processing
detectCores() # How many cores do I have?
num.cores <- 10 ## Don't all the cores. Leave a few to do other things.

## Set working directory
setwd("C:/temp/topology_pro")
```

Next, we will create an `sf` object containing the streams data, set the LSN name, and build the initial LSN using `lines_to_lsn`. In this case, we are importing a shapefile, but there are no constraints on the data format, except that it must have `LINestring` geometry.

Checking topology using `lines_to_lsn` is optional and there are a few arguments that must be set to do so:

- **check_topology:** Logical indicating whether the topology should be checked;
- **snap_tolerance:** Distance in map units. Two nodes separated by a distance less than or equal to `snap_tolerance` are assumed to be connected.
- **topo_tolerance:** Distance in map units. Two nodes separated by a distance less than or equal to `topo_tolerance` are flagged as potential topological errors in the network.

Run all the code listed below:

```
## Import the streams dataset as an sf object
river_net<- st_read("ARiver.shp")

## Set output name for LSN
lsn_path1<- "c:/temp/topology_pro/work/lsn1"

## Cast the streamline shapefile as linestring format
river_net2 = st_cast(river_net,"LINESTRING")

|
## Build the initial LSN and check the topology
edges<- lines_to_lsn(
  streams = river_net2,
  lsn_path = lsn_path1,
  snap_tolerance = 1,
  check_topology = TRUE,
  topo_tolerance = 20,
  overwrite = TRUE,
  verbose = TRUE,
  remove_ZM = TRUE,
  use_parallel = TRUE,
  no_cores = num.cores)
```

The arguments `snap_tolerance` and `topo_tolerance` are both provided in map units. It is generally a good idea to set the `snap_tolerance` to a relatively small value compared to the `topo_tolerance` argument. Nodes separated by a Euclidean distance $\leq \text{snap_tolerance}$ are assumed to be connected. If this distance $\leq (\text{snap_tolerance}/10000)$, the nodes are automatically snapped when `check_topology = TRUE`. When $(\text{snap_tolerance}/10000) \leq \text{distance} \leq \text{snap_tolerance}$, the nodes are not snapped and are instead flagged as potential errors for the user to check. Similarly, when $(\text{snap_tolerance}) < \text{distance} \leq \text{topo_tolerance}$, nodes are flagged as potential errors. To learn more about the other function arguments, simply type `help(lines_to_lsn)`.

The function `lines_to_lsn` always saves five output files to `lsn_path` when it runs successfully. These include edges and nodes in GeoPackage format (.gpkg) and three .csv files containing tables that describe the topological relationships in the LSN (`nodexy.csv`, `noderelationships.csv`, and `relationships.csv`). If potential topological errors are found in the network, there will be an additional file named `node_errors.gpkg`. The LSN is comprised of the six files listed below.

Name	Date modified	Type
 edges.gpkg	4/25/2024 2:03 PM	GPKG File
 node_errors.gpkg	4/25/2024 2:03 PM	GPKG File
 noderelationships.csv	4/25/2024 2:03 PM	Microsoft Excel Comma...
 nodes.gpkg	4/25/2024 2:03 PM	GPKG File
 nodexy.csv	4/25/2024 2:03 PM	Microsoft Excel Comma...
 relationships.csv	4/25/2024 2:03 PM	Microsoft Excel Comma...

The contents of the new folder can also be checked from R:

```
> ## Check output files. If node_errors.gpkg exists, then there are
> ## potential errors to check
> list.files(lsn_path1)
[1] "edges.gpkg"          "node_errors.gpkg"      "nodereationships.csv"
[4] "nodes.gpkg"          "nodexy.csv"           "relationships.csv"
```

In this case, node_errors.gpkg exists. Run the following code to look at and summarize the types of topological errors and restrictions identified:

```
> ## Import node errors and format columns
> node_errors <- st_read(paste0(lsn_path1, "/node_errors.gpkg"),
+                         quiet = TRUE) %>%
+   modify_if(is.character, as.factor)
>
> ## Summarise
> summary(node_errors)
```

pointid	nodecat	error	geom
Min. : 275.0	Confluence:7	Complex Confluence :2	POINT :13
1st Qu.: 589.5	Outlet :6	Converging Node :2	epsg:5070 : 0
Median : 865.5		Dangling Node :4	+proj=aea ... : 0
Mean : 879.8		Downstream Divergence :2	
3rd Qu.:1327.5		Intersection Without Node:1	
Max. :1338.0		Unsnapped Node :2	
NA's :1			

The object node_errors contains three columns in addition to the geom (or geometry) column that will be useful for checking topology errors.

- **pointid:** point identifier for each node.
- **nodecat:** node category, which can take on values of Source, Confluence, Pseudonode, and Outlet.
- **error:** node error type, which includes topological errors (Unsnapped Node, Dangling Node, and Intersection Without Node) and restrictions (Complex Confluence, Converging Node, and Diverging Node).

Note that the pointid and nodecat columns are also found in nodes.gpkg. It is also important to recognize that node_errors indicate *potential* topological errors. The user must verify whether these are true errors and then edit the topology accordingly. At this point, we move to ArcGIS Pro where the topological editing will occur.

At this point in the exercise, you should have now run all the lines of the R code in the first section of the script, up through this command:

```
## Summarise
summary(node_errors)
```

You should see data in your **\work** folder.

5. Verifying and correcting topological errors in ArcGIS Pro

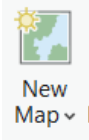
We assume that the reader has a working knowledge of ArcGIS Pro and is familiar with ArcGIS Pro editing tools. For those that need additional help, the ArcGIS Pro documentation is a good place to start.

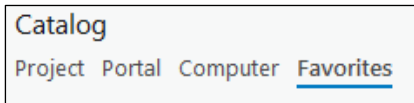
Open ArcGIS Pro. At the initial window, choose Start without a template.

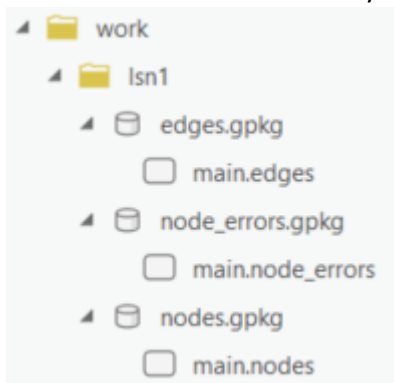


Start without a template

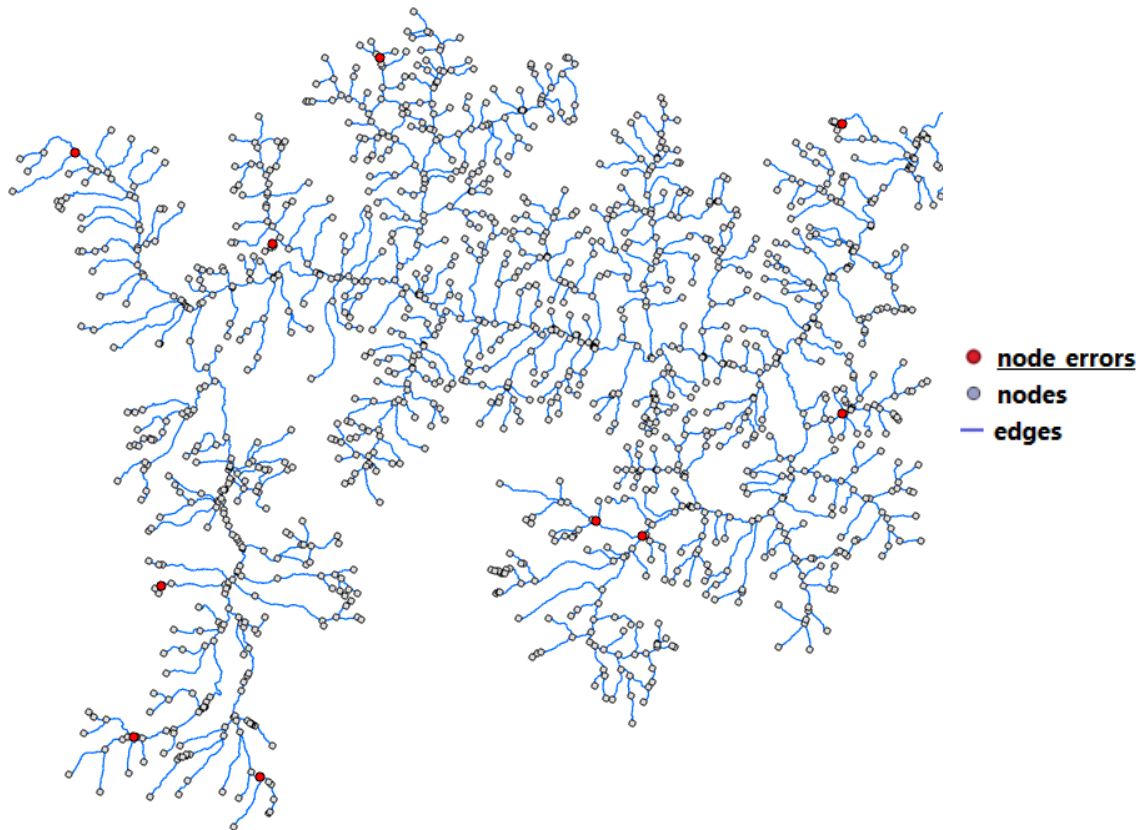
- a. Click the New Map feature:



- b. Choose View > Catalog Pane
- c. In the Catalog Pane, select Favorites, and right click in the pane. Choose Add Folder.
A screenshot of the Catalog Pane in ArcGIS Pro. It shows tabs for 'Project', 'Portal', 'Computer', and 'Favorites'. The 'Favorites' tab is selected and underlined.
- d. Navigate to the C:\temp\topology_pro folder and hit OK.
- e. In the Catalog Pane, navigate to C:\temp\topology_pro\work\lsn1 folder.
- f. Open the edges.gpkg, nodes.gpkg, and node_errors.gpkg containers. These are GeoPackages generated from the R code and are part of the LSN. GeoPackages are an open-source GIS data model that can be read by ArcGIS.



- g. Drag the main.edges, main.node_errors, and main.nodes feature classes into the map viewer.
- h. Modify the symbology as you see fit.



- i. Save the ArcGIS project. Project > Save Project As. Navigate to the folder: C:\temp\topology_pro. Save as Name: topology_pro_project.

5.2 Check the nodecat values

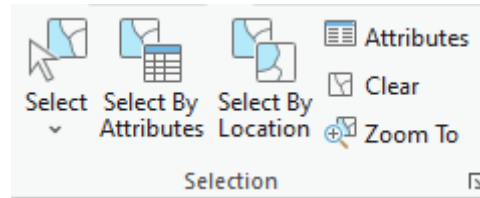
The first step should always be to check the nodes (not node_errors) on the network. In particular, the number and positions of Outlet nodes should be investigated to identify obvious errors. For example, having many outlets may indicate the presence of topological errors, especially when they are found in the interior of a stream network. The extra, erroneous Outlets are usually the result of a failure to snap the end nodes of edges segments (e.g. Unsnapped Nodes or Dangling Nodes).

1. Uncheck main.node_errors so it isn't visible.
2. Open the nodes attribute table by right-clicking main.nodes in the Contents panel and clicking Attribute Table.
3. Select the rows of the attribute table where the "nodecat" = "Outlet". There should be 8 Outlet nodes selected.

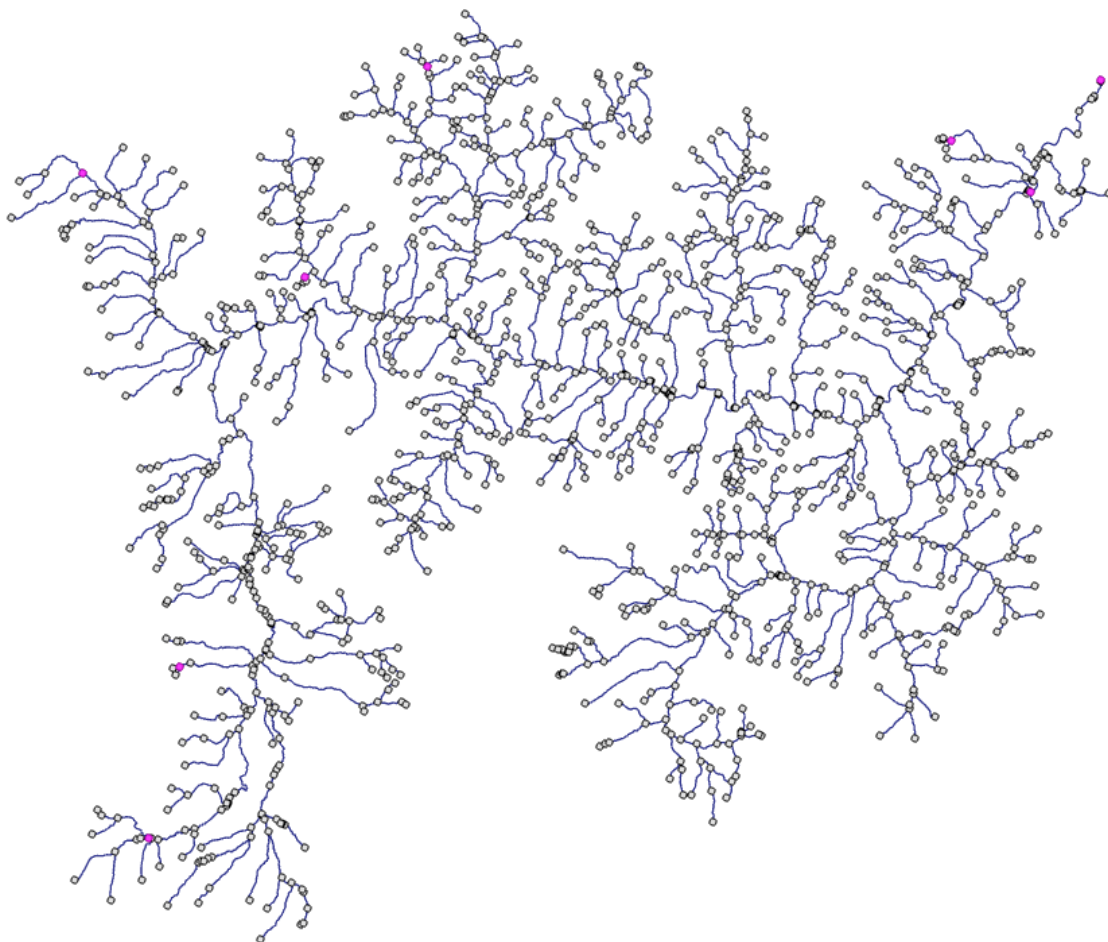
Selection: Select By Attributes

Where	nodecat	is equal to	Outlet	
-------	---------	-------------	--------	--

4. Close the attribute table. The selected Outlet nodes are highlighted here in pink below, but the selection colour may vary depending on user default settings. To change the highlight color for selections, choose the arrow in the lower right of the selection group.



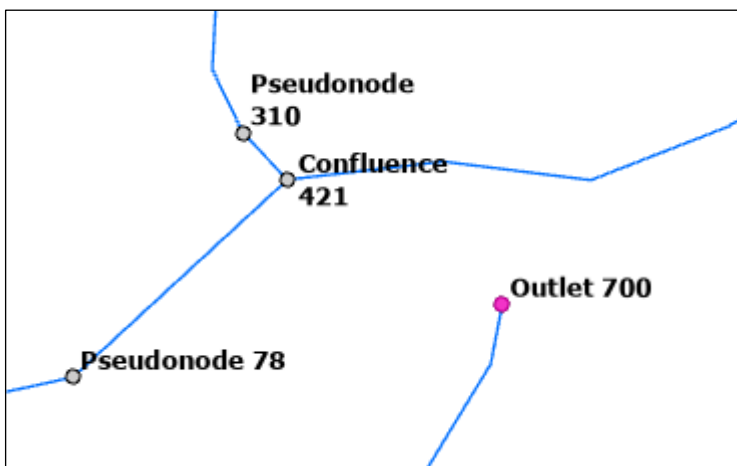
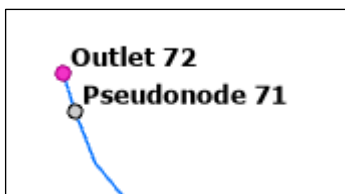
The selection should look like this:



There are only 2 true outlets in this dataset, but 8 Outlet nodes have been identified (pink). Notice the Outlets highlighted in the interior of this stream network. These are unlikely to be true outlets. Zooming in on one of the selected Outlets (pointid = 1322) reveals a situation like the one below, where a node has been identified as an outlet because it has not been snapped to the neighbouring stream segment.



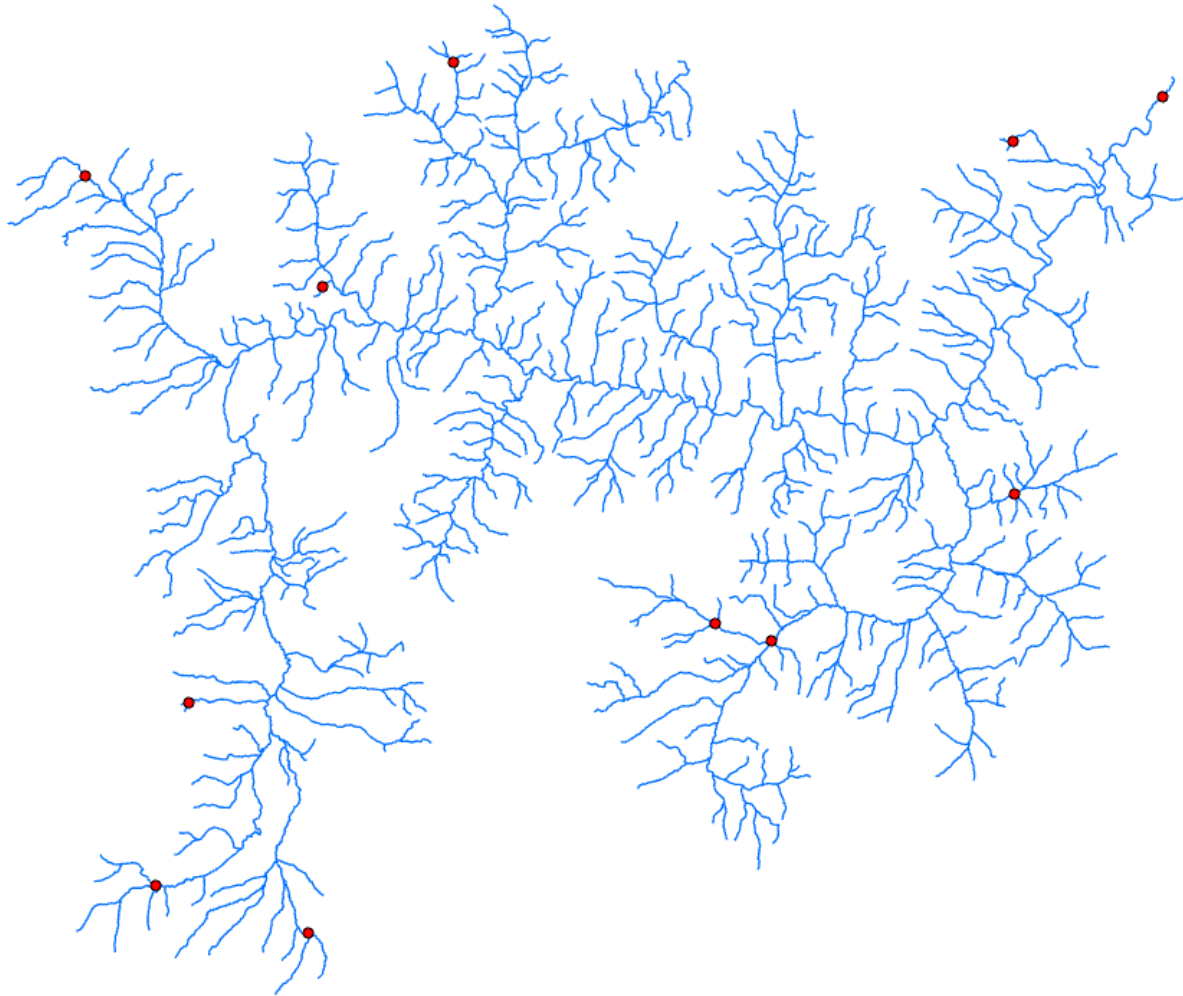
5. Investigate the Outlet nodes. For large datasets with many Outlets, manual inspection of every node will not be feasible. In this case, simply check a few at random, but pay special attention to any Outlets located in the interior of the network.
6. The true outlets are features with pointid 72 and 700. Try to find them by selecting each in the attribute table and zooming in.



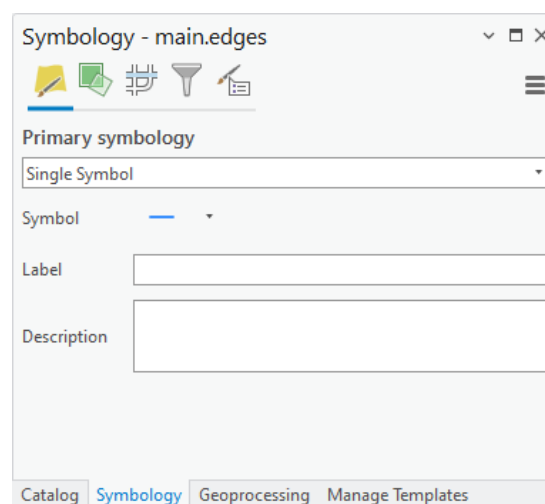
7. Remove nodes from the Contents Panel because it will not be needed again for this tutorial. However, keep in mind that the number and positions of Outlets should be checked every time an LSN is generated by lines_to_lsn.

5.3 Edit various topology errors

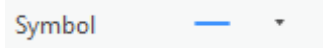
1. Turn on node_errors in the Contents Panel (if it is turned off). The Map view should look something like this, where points represent node_errors and lines represent edges.



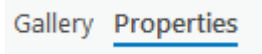
2. Change the symbology of edges to lines capped with arrows indicating the digitized direction and direction of flow in the LSN.
 - a. Right-click on main.edges in the Contents panel and choose Symbology. This will open the Symbology window where the symbology for edges can be changed.
 - b. Click on Symbology tab at the bottom if necessary.



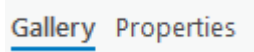
- c. Single click the blue line here, to the right of Symbol.



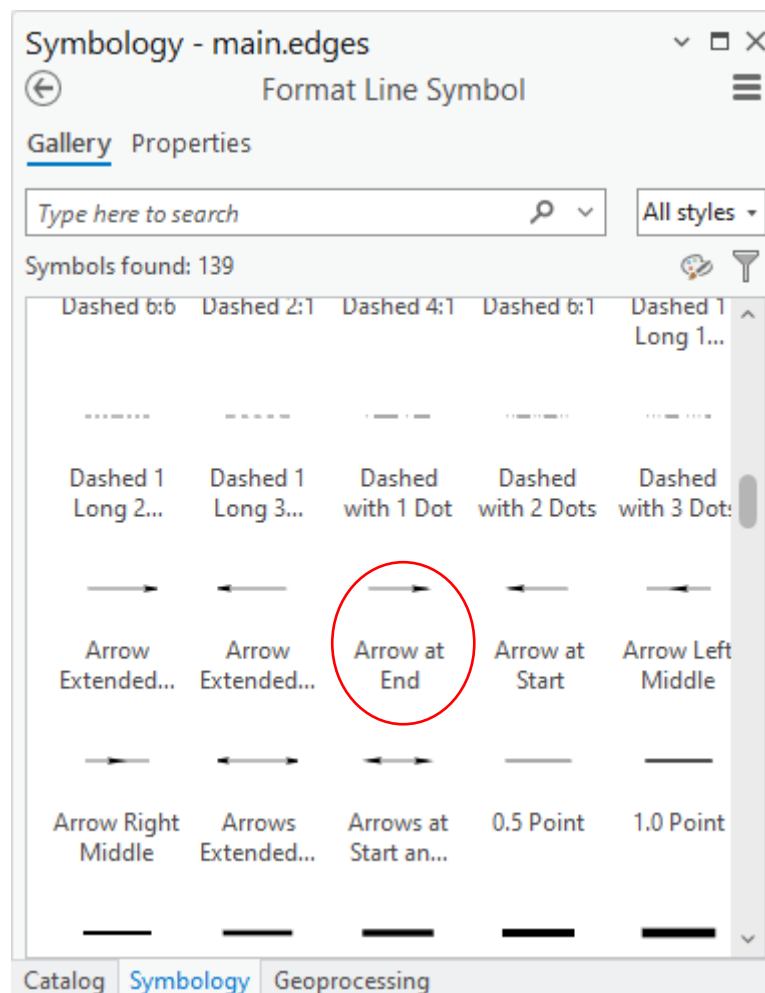
Change the window view from Properties



To Gallery view



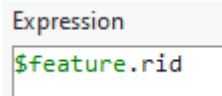
Scroll down to find Arrow at End



Click the Arrow at End options to redraw the streamline symbology.

3. Prepare to edit the Complex Confluence.

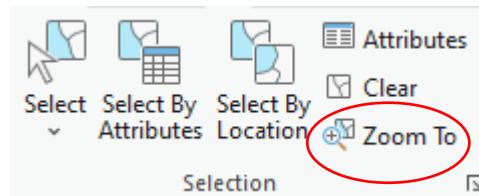
- a. First label the edges with the rid field. Right click main.edges and click Label to enable labels. Right click again and select Labeling Properties. Under Expression, type in, \$feature.rid, and hit Apply.



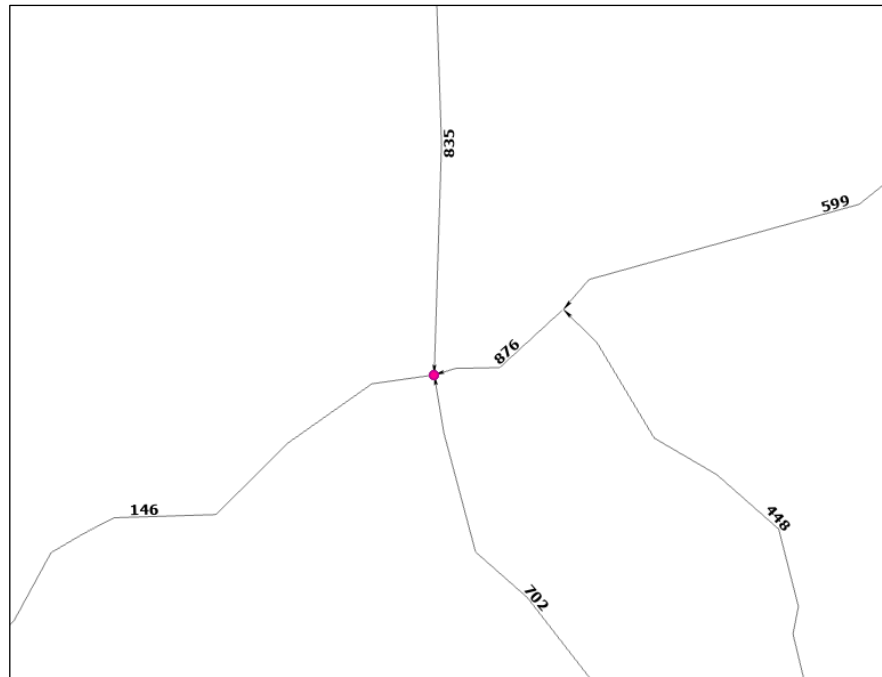
- b. Right click on node_errors in the Contents Panel and click Attribute Table, to select the first node error.
- c. The node_errors include two Complex Confluences. Select this record:

3	3	Point	275	Confluence	Complex Confluence
---	---	-------	-----	------------	--------------------

- d. Close the attribute table and zoom into the node_error highlighted in magenta. You may have to temporarily turn off the edges in the Layers panel to see it. You can zoom to selected features using the Zoom to function in the Selection group.

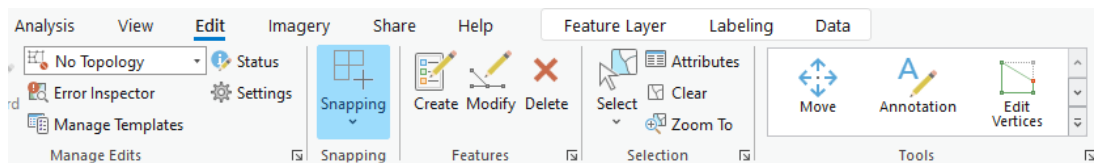


In the image below, the magenta point represents the complex confluence, and the numbers are the **rid** values for each edge. Notice there are three upstream edge features flowing into the node (rid = 702, 835, and 876) and one downstream edge flowing out (rid = 146). Complex confluences are not allowed and so one of the upstream edges must be edited. If one of the upstream edges is a small headwater stream, it may be appropriate to simply delete it. If not or there is another reason to retain all three edges, the end node of one upstream edge must be moved slightly up or downstream of the of the complex confluence. We describe how to do that next.

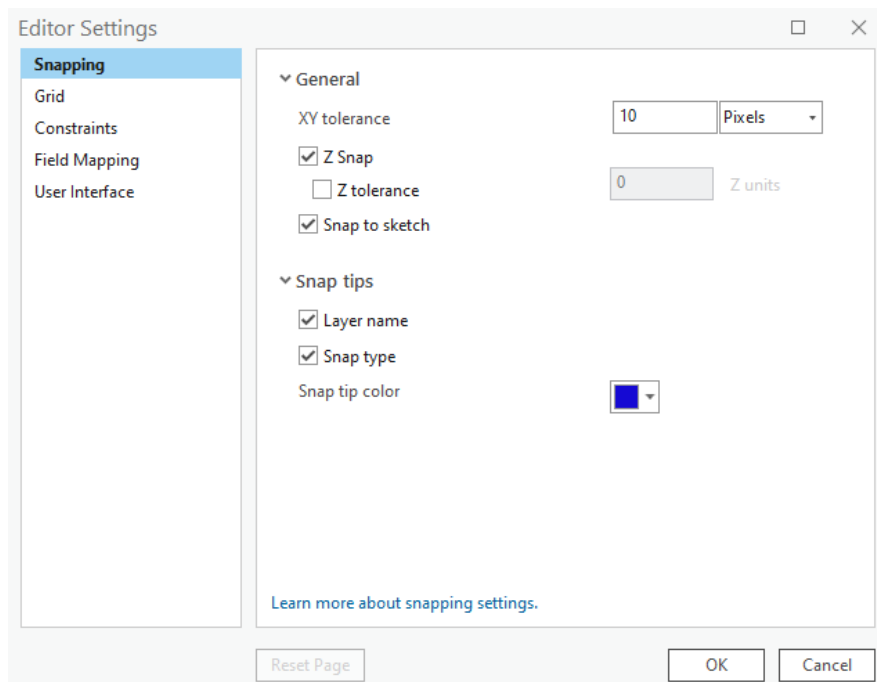


4. Edit Complex Confluence.

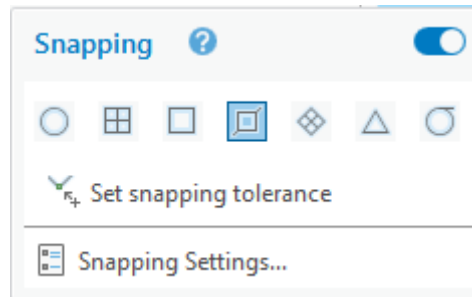
- First set the snapping parameters. Select Edit to reveal the editing ribbon.



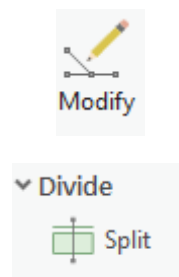
- Click Snapping at the drop-down arrow and then click Snapping Settings. Be sure that XY tolerance is set at 10 pixels.



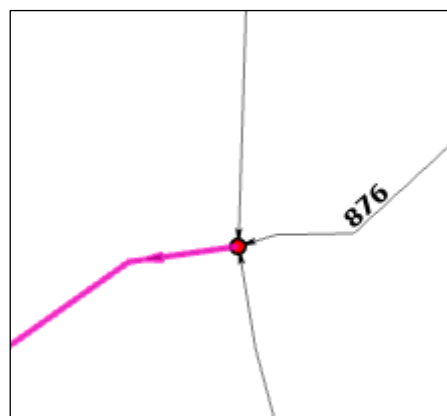
- c. Hit OK to close out. Click Snapping again at the drop-down arrow to reveal the feature snapping options. Select the center feature for snapping to the nearest edge. Also, be sure that the slider is positioned to the right to ensure snapping is turned on.



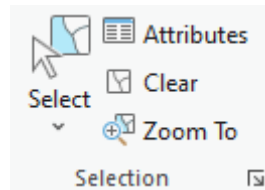
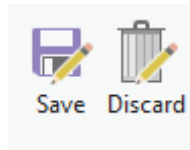
- e. Next, we want to split line 146. After doing so we will snap the end point of line 835 to that split location. This procedure will eliminate the single junction with three tributaries and create two new junctions, each with only two tributaries. To split line 146, choose Modify, then Split.



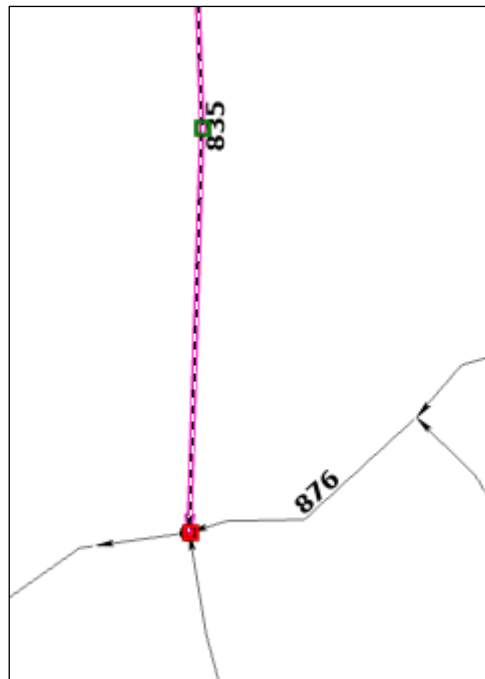
- f. Select line 146 by clicking on it. Then hover over the line with the cursor until it snaps to a location about 50 meters downstream from the junction. When the line is split, a new arrowhead will appear on the streamline indicating that a new line segment has been created.



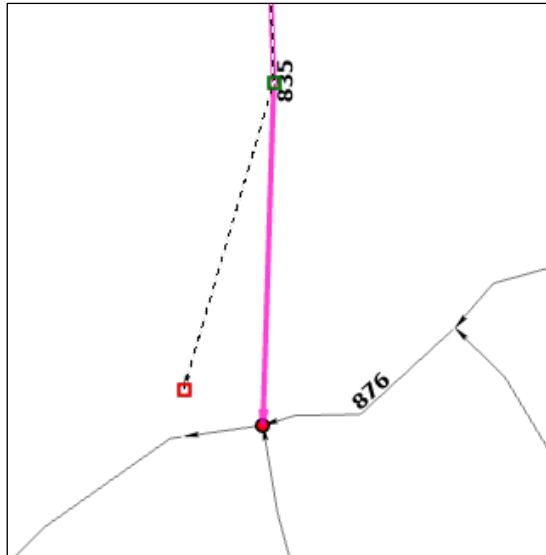
Click the Save button on the editing ribbon. If you would like to discard the change, click Discard. Agree to save all edits and clear the selection using the Clear button.



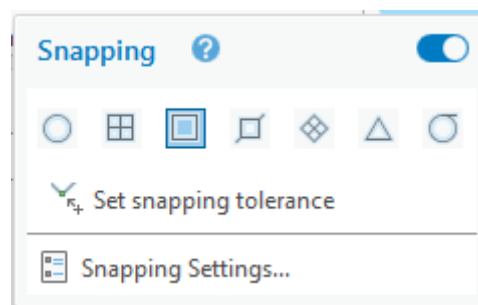
- g. Now we will move the end point of line 835 to the split location on line 146. While in the Edit ribbon, select line 835. Right click the line and choose Edit Vertices at the bottom of the pull down.



Hover over the endpoint until a diamond appears. Then click and drag the end point away from the junction.



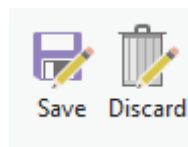
Now change the snapping option to snap at the vertex. Be sure to turn off edge snapping.



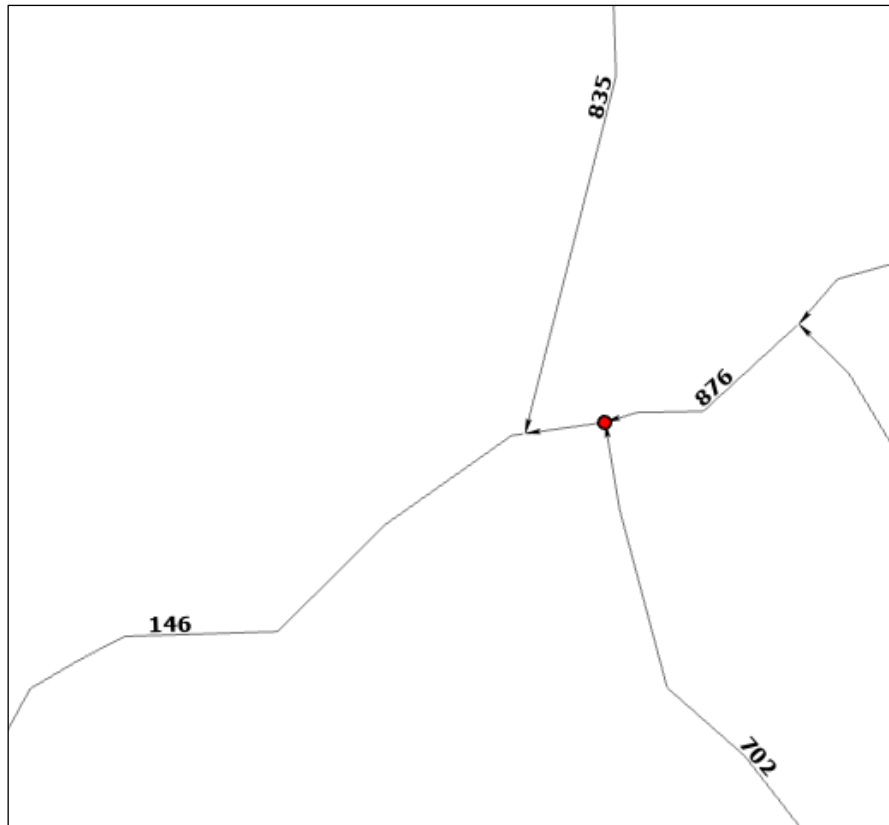
Be sure line 835 is selected. Hover over the end point again to select it and move the end down to the split location until it snaps at the new vertex you created. Click the green check box to accept the change.



- h. Unselect all features and save the edits with the Save button.



- i. The final line work should look like this.

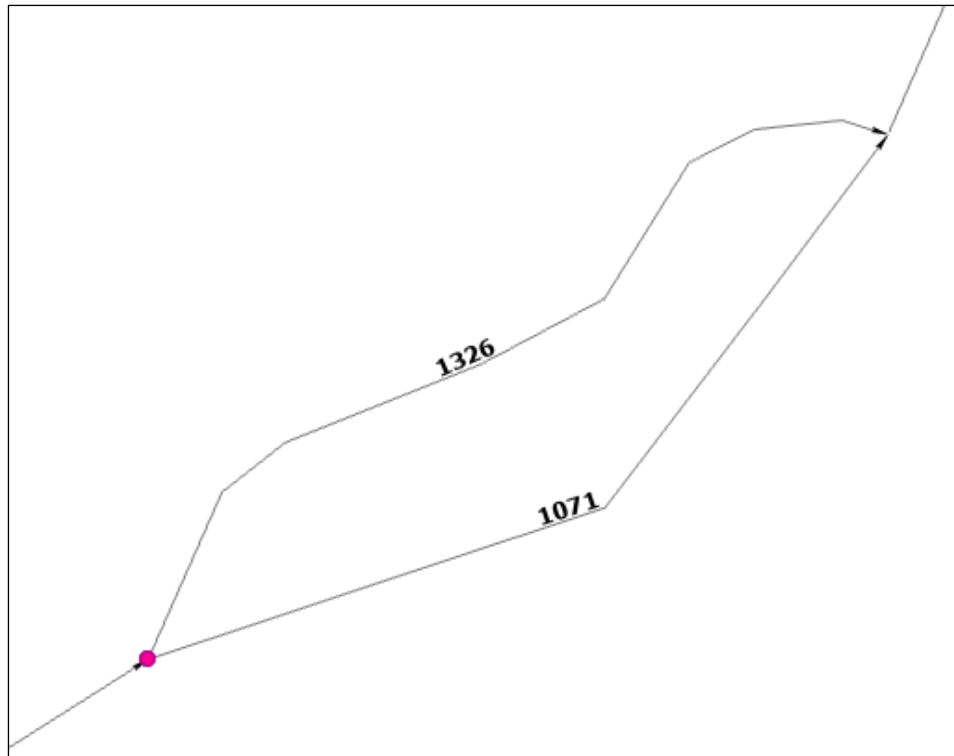


5. Edit Downstream Divergences.

- Open the node_errors attribute table and use Select by Attributes to select Downstream Divergence.

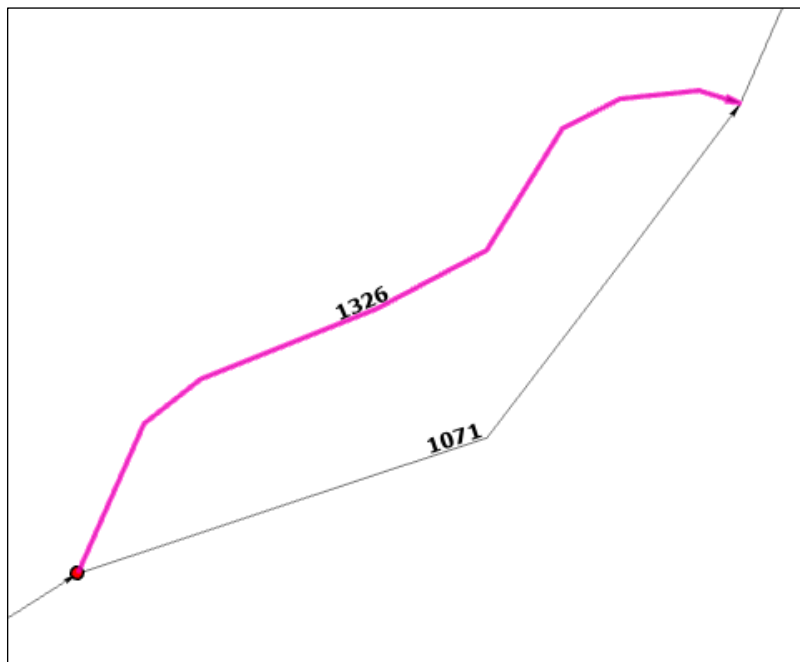
Where	error	is equal to	Downstream Divergence	X
-------	------------------------------------	-------------	--	----------------------------------

- There are two downstream divergences that are caused by different characteristics of the line work. First go to the error to the northeast, pointid = 627, shown below.

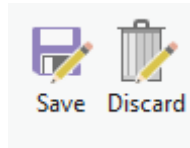
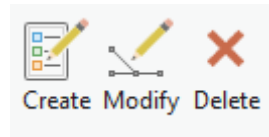


This feature represents a braided channel. If you query the two channels, you'll find that channel 1326 doesn't have a valid drainage area (TotDASqKM) or other features, compared to 1071. We'll delete reach 1326 so the channel is no longer braided in the digital representation.

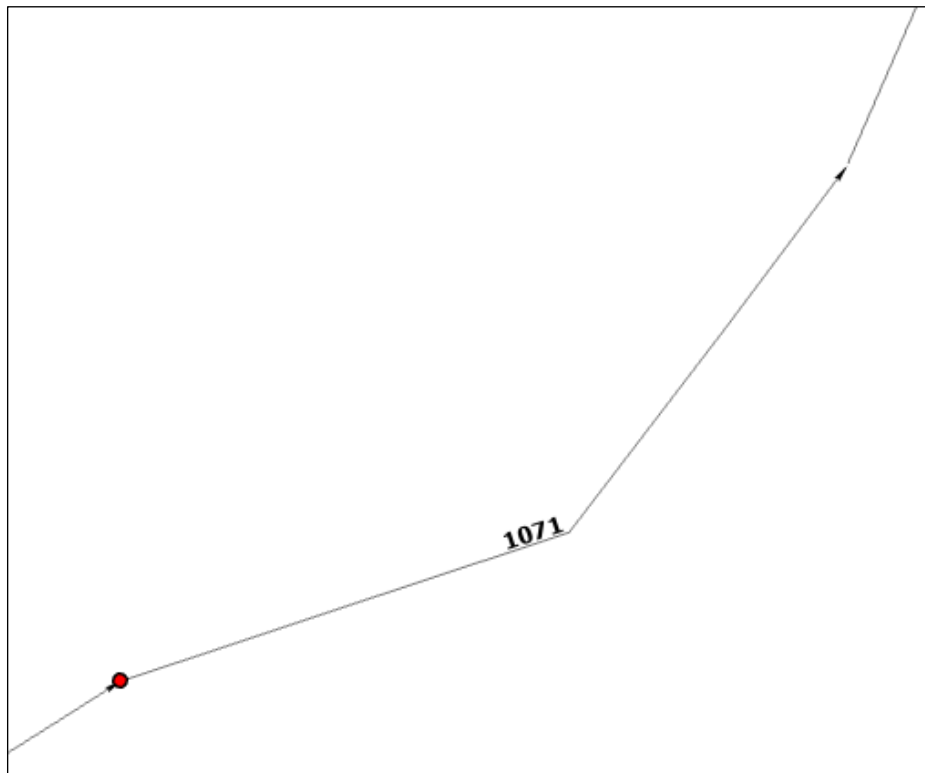
- c. Activate the editing ribbon by selecting Edit. Then Select reach 1326.



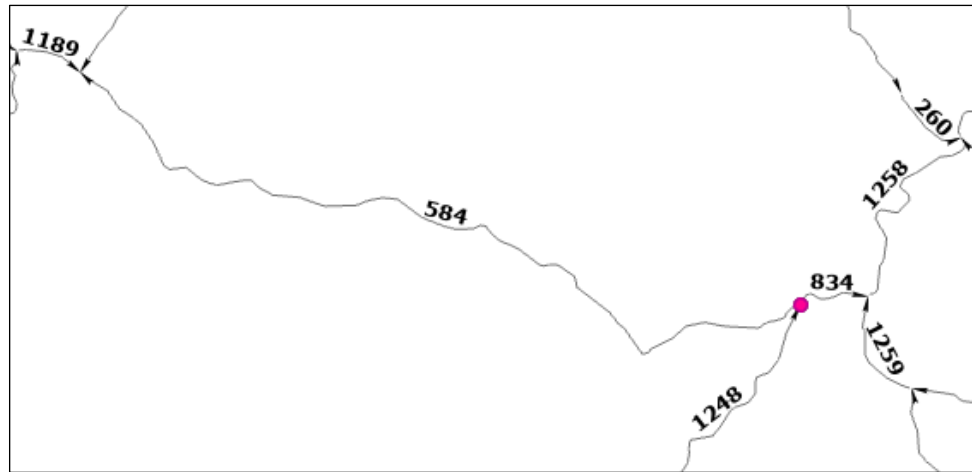
- d. Hit the Delete button on the edit ribbon, then Save edits.



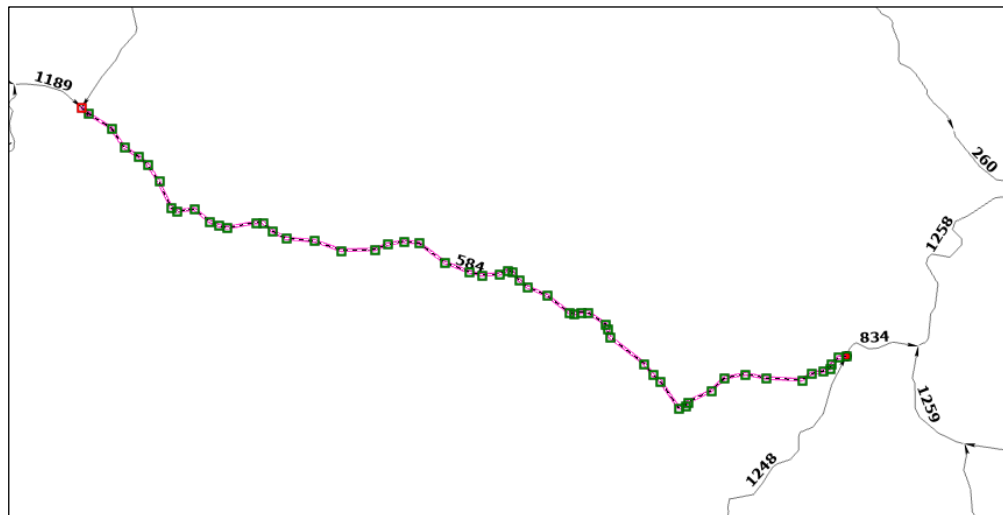
- e. This Downstream Divergence has been corrected and should look like the graphic below.



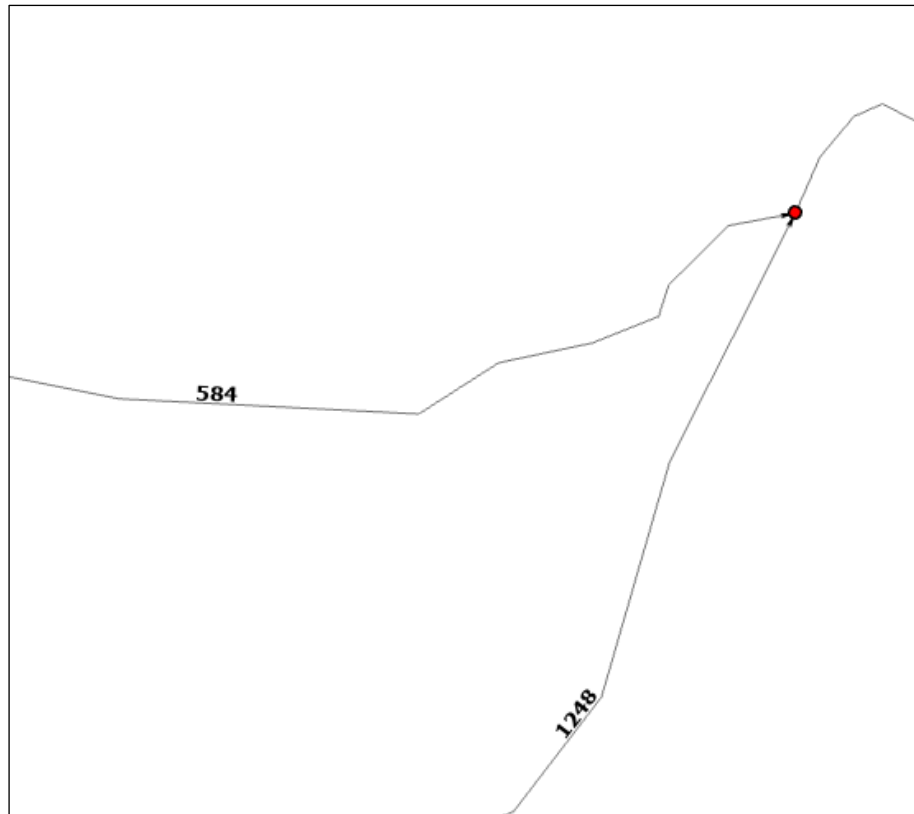
- f. Next, we'll edit the Downstream Divergence where pointid = 865. Open the attribute table for main.node_errors and select pointid = 865. Zoom to that location and look at the flow direction on reach 584. You'll see that the arrowhead is pointing in the wrong direction because the digital segment was digitized in the wrong direction. This problem is causing a Downstream Divergence, where reach 1248 seems to split and flow into 584 and 834. We'll correct this issue by flipping the direction of 584.



- g. Enable the Edit ribbon and select reach 584. Right click the reach and choose Edit Vertices.



- h. After the vertices are displayed, again right click on the selected reach, but do so at a part of the line without vertices (green boxes) so that the cursor isn't snapped to a vertex. Click Reverse Direction. Then click the green check box at the bottom of the screen and Save your edits. Clear the selection and you'll see that reach 584 is now pointing in the downstream direction. This Downstream Divergence is now corrected.



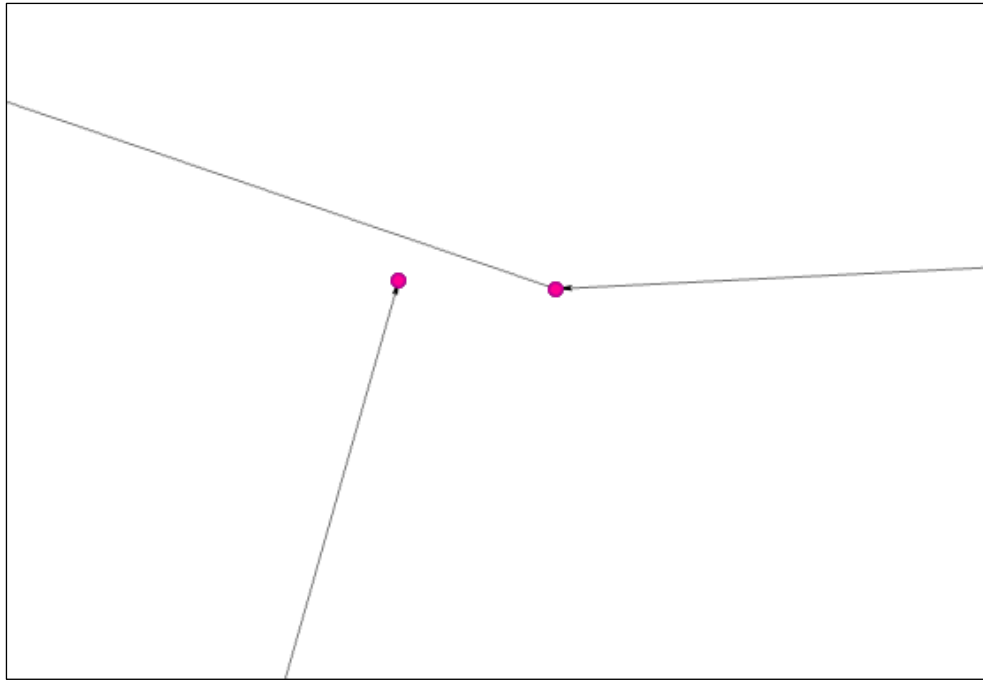
6. Correct Unsnapped Nodes.

- Unsnapped nodes are two nodes that are within the snap tolerance of the `lines_to_lsn` function shown here.

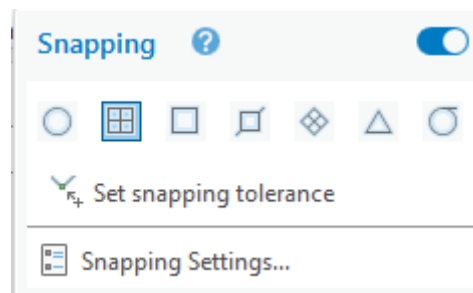
```
## Import the streams dataset as an sf Object
river_net<- st_read("ARiver.shp")

## Build the initial LSN and check the topology
lsn_path1<- "c:/temp/topology_pro/work/lsn1"
edges<- lines_to_lsn(
  streams = river_net,
  lsn_path = lsn_path1,
  snap_tolerance = 1,
  check_topology = TRUE,
  topo_tolerance = 20,
  overwrite = TRUE,
  verbose = TRUE,
  remove_ZM = TRUE)
```

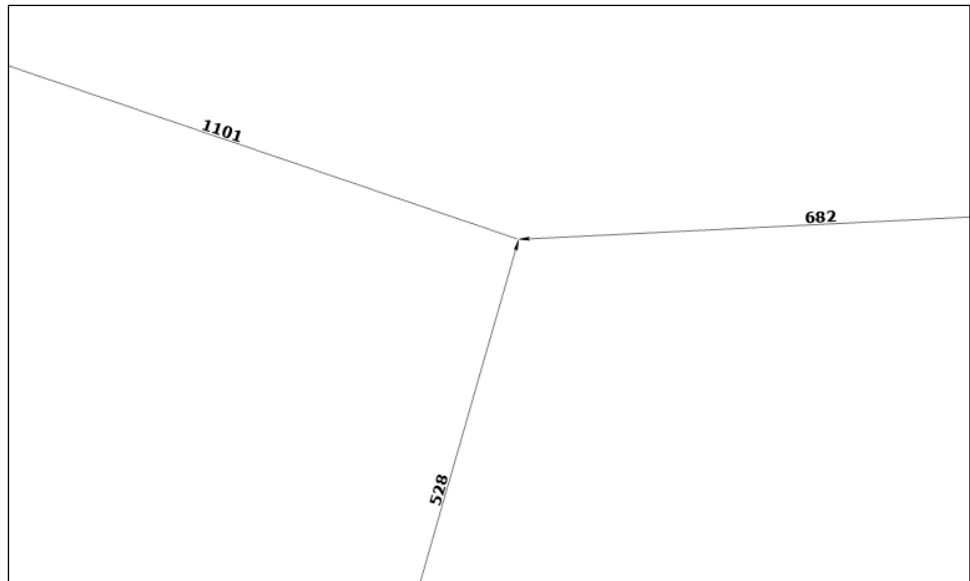
- Open the attribute table for `main.node.errors` and select `error = Unsnapped Node`. Two nodes will be selected that are very close to each other (less than 1 meter). Zoom in until you can see both nodes.



- c. Turn off the main.node_errors layer in the Contents pane. Click Edit to make the edit ribbon visible. Set snapping for endpoints.

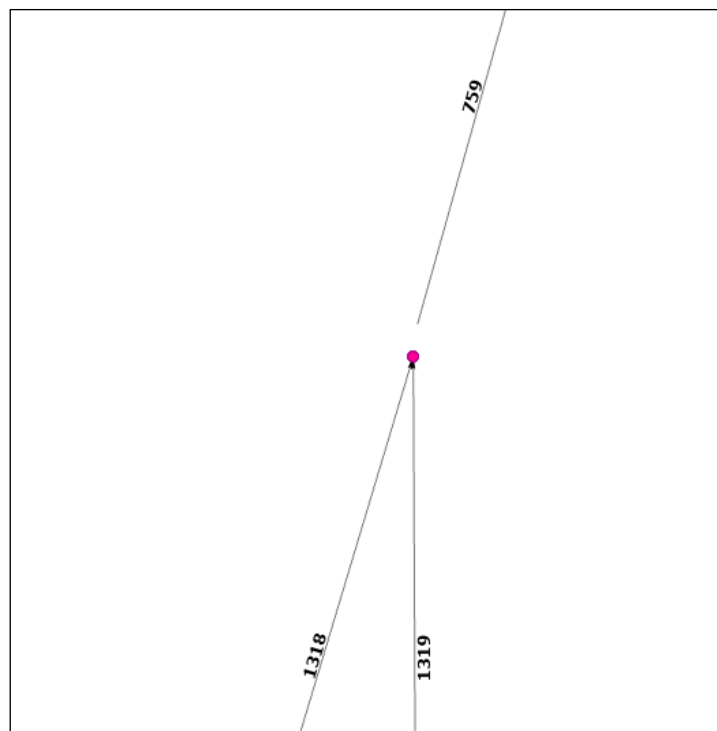


- d. Select reach 528. Hover over the endpoint, then click and drag to the other node location to the northeast until they snap. Unselect and Save edits. Both Unsnapped Node errors are now corrected.



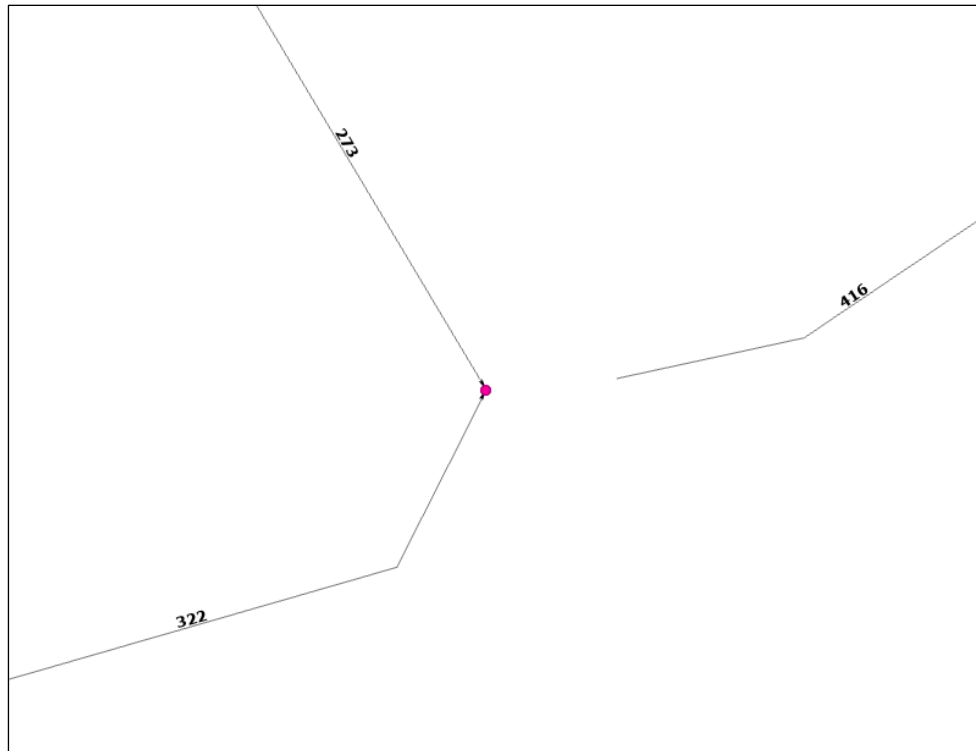
7. Correct Converging Nodes.

- a. There are two node errors classified as Unsnapped Nodes. pointid = 477 and 1329. Both can be edited using the same technique. Converging Nodes flag a location where two tributaries come together, but do not continue the flow path to a downstream reach.



- b. Zoom to the node error where pointid = 1329. Turn off the main.node_errors in the Content pane. Activate the Edit ribbon and make sure snapping is still set to endpoints.

- c. Select reach 759. Right click and select Edit Vertices. Hover over the endpoint and click and drag to the confluence of 1318 and 1319, until the endpoints snap. Unselect and Save the edit.
- d. Follow the same procedure for the next Converging Node. Zoom to the node error where pointid = 477.



- e. Turn off node errors, select reach 416, then snap the endpoint to the intersection of 273 and 322. Save edits.

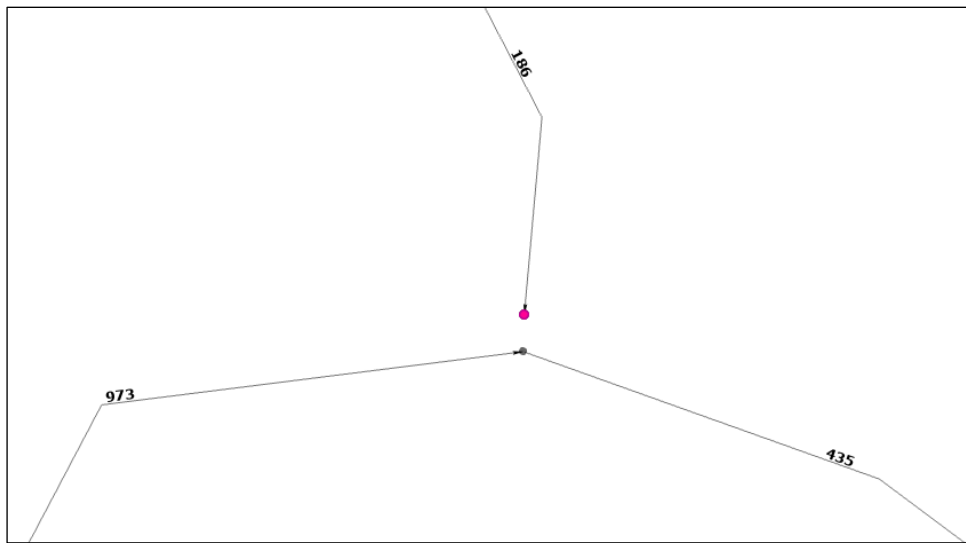
8. Correct Dangling Nodes.

- a. Dangling nodes are not necessarily errors but should be checked to make that determination. Dangling nodes occur where a node and an edge are in close proximity and less than the distance specified by the topo_tolerance value in the lines_to_lsn function. In this case, 20 meters as shown below. There are four Dangling Nodes to edit in this dataset.

```
## Import the streams dataset as an sf Object
river_net<- st_read("ARiver.shp")

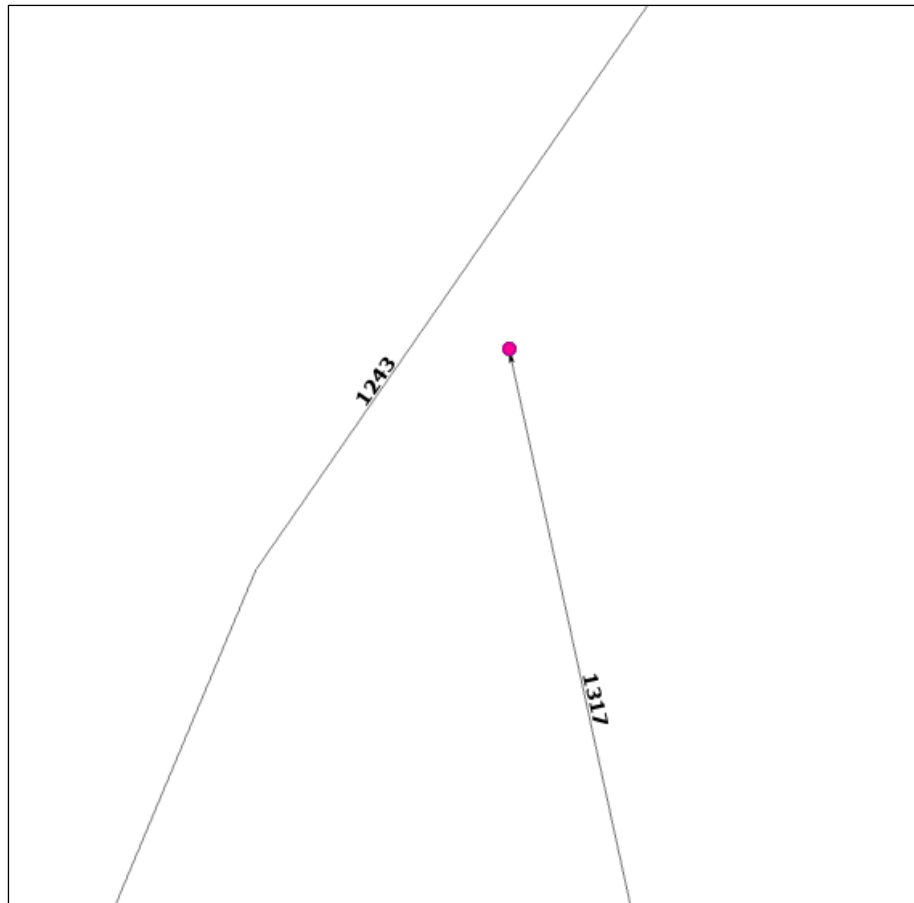
## Build the initial LSN and check the topology
lsn_path1<- "c:/temp/topology_pro/work/lsn1"
edges<- lines_to_lsn(
  streams = river_net,
  lsn_path = lsn_path1,
  snap_tolerance = 1,
  check_topology = TRUE,
  topo_tolerance = 20,
  overwrite = TRUE,
  verbose = TRUE,
  remove_ZM = TRUE)
```

- b. Dangling Node 1. Turn on the node errors, nodes, and edges. Open the main.node_errors attribute table and select the point with pointid = 344. Zoom to that location. The dangling node and pseudo node are approximately 3 meters apart.

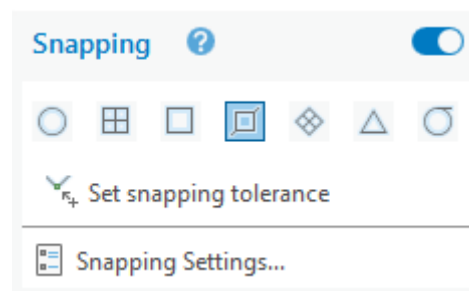


In this case we'll snap the end node on 186 to the pseudo node joining 973 and 435. Turn off the node errors and nodes. Activate the Edit ribbon. Select reach 186. Right click and choose Edit Vertices. As previously, click on the end node and pull it down to the pseudo node until the two snap together. Unselect and Save edits.

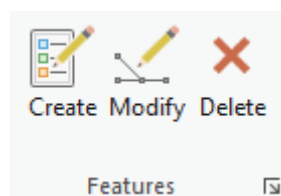
- c. Dangling Node 2. In the node errors feature class, zoom to pointid = 1327. This dangling node is approximately 4 meters from the nearest edge. In this case we'll split reach 1243 and then snap reach 1317 at the split location.



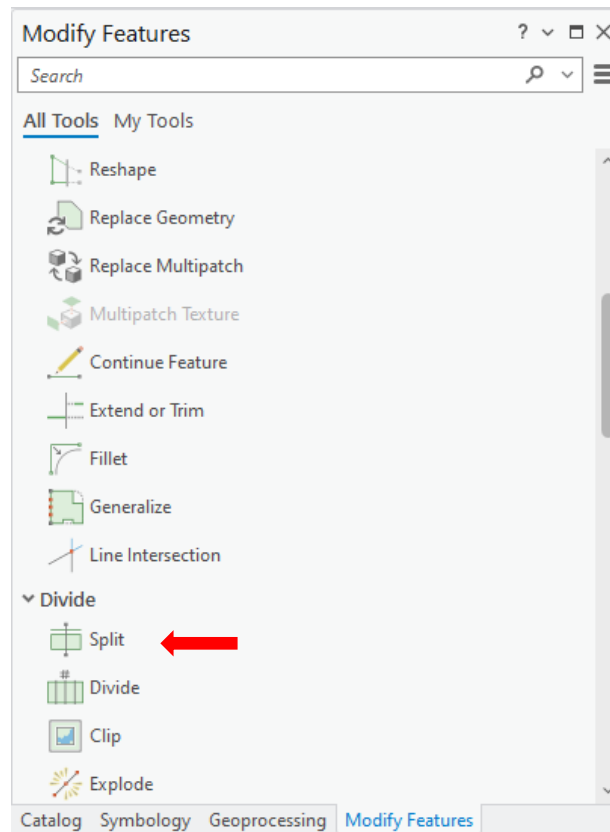
Turn off the node errors and nodes feature classes. Activate the Edit ribbon and set snapping to edge snapping.



Select reach 1243. Select Modify from the edit ribbon.



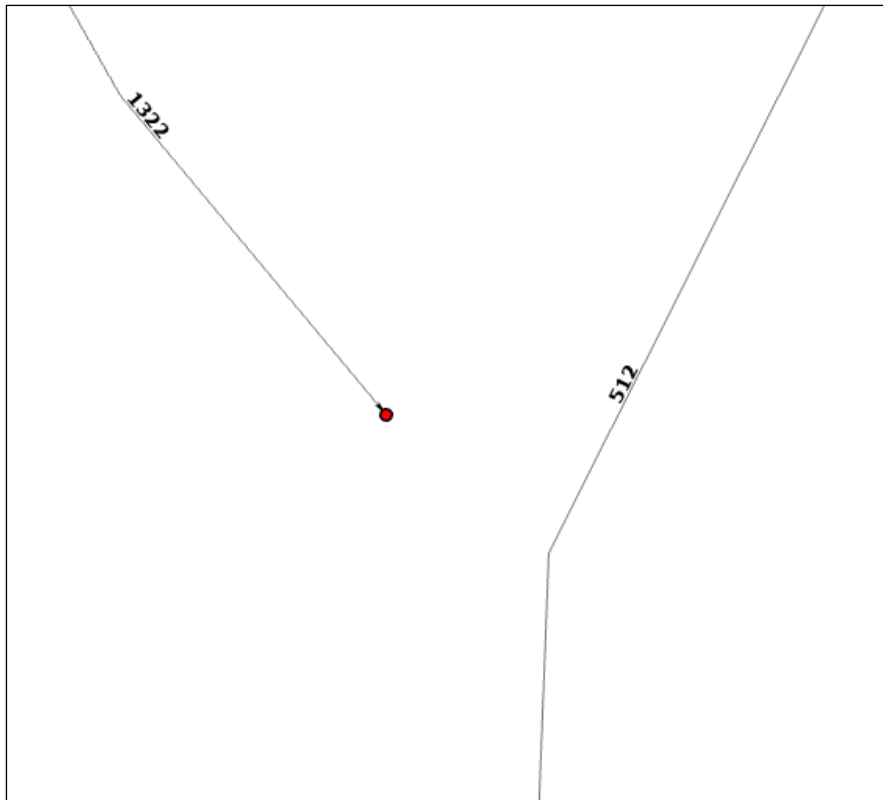
The Modify Features dialog should appear. Scroll down and select the split option.



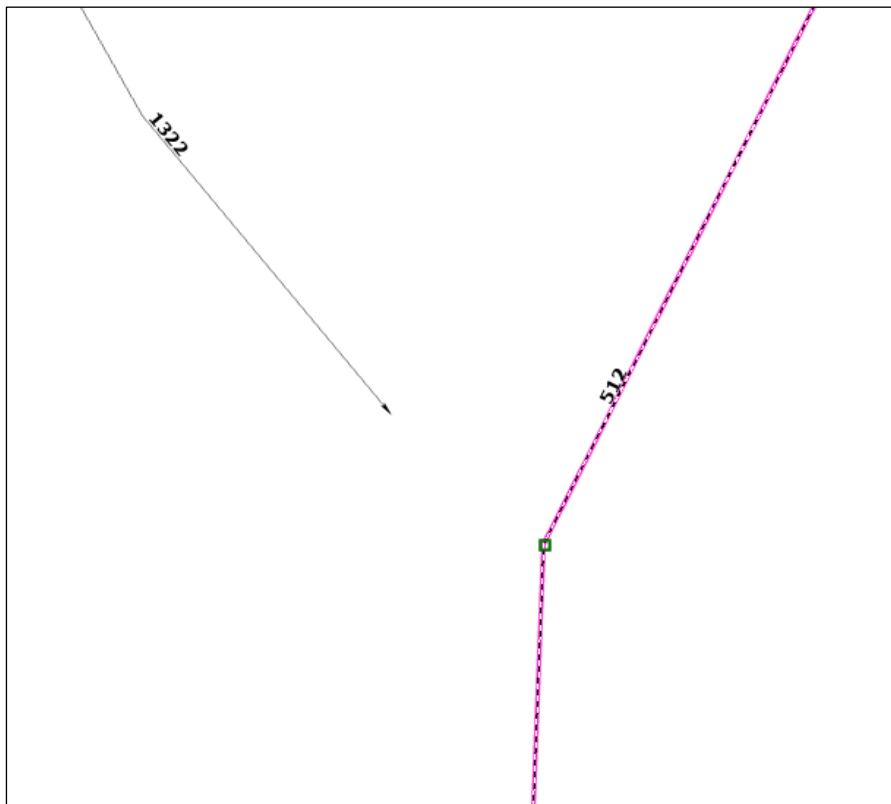
Split the selected reach at the location closest to the dangling node. Select the green check box.

Change snapping to end point and turn off edge snapping. Select reach 1317. Right click and choose Edit Vertices. Select the end point of 1317 and snap it to the split location. Hit the green check mark and unselect. Save edits.

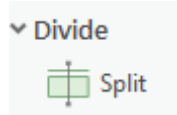
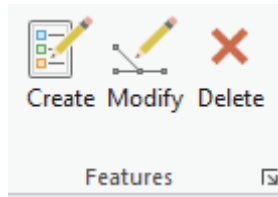
- d. Dangling Node 3. Open the attribute table for main.node_errors, select pointid = 1334 and zoom to the location. In this case we'll snap the endpoint of reach 1322 to the closest vertex on reach 512.



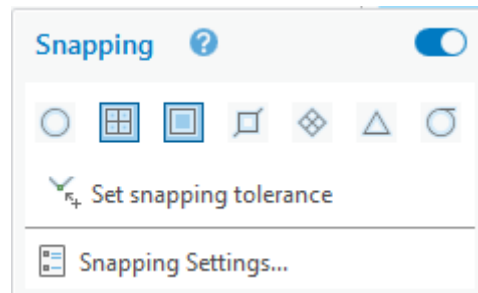
Turn off node errors in the Contents pane. Select reach 512, then right click and choose Edit Vertices. We'll be splitting 512 at the green box (vertex) shown below.



Use the procedures we've used previously to split reaches. Edit ribbon > Modify > Split.

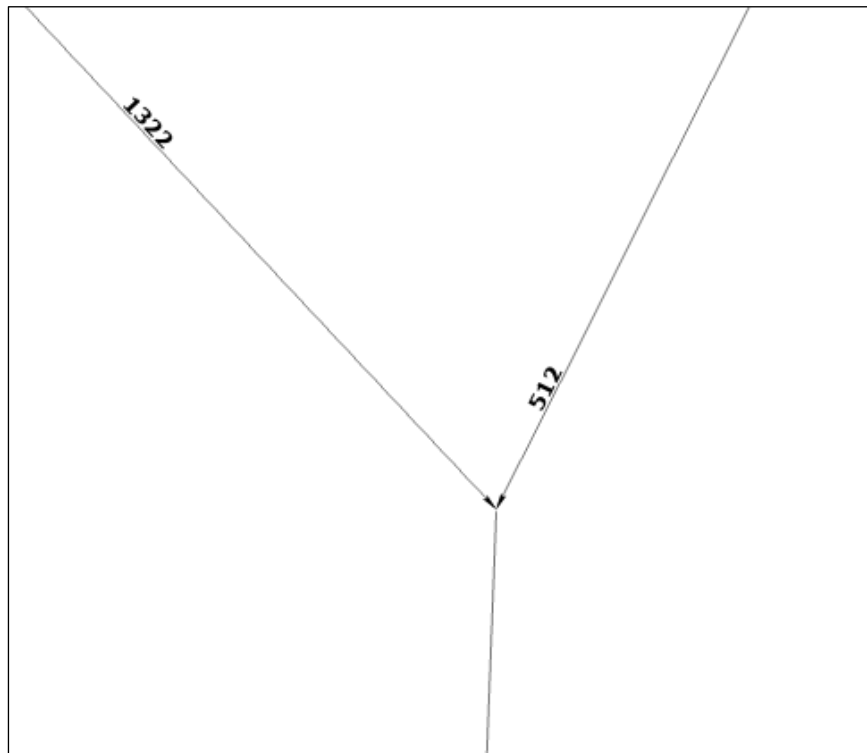


After splitting at the vertex, hit the green check to accept the edit. Next, be sure the snapping options should look like this.



Select reach 1322, right click and select Edit Vertices. Click and hold the endpoint and snap it to the new split location. Clear the selection and Save edits.

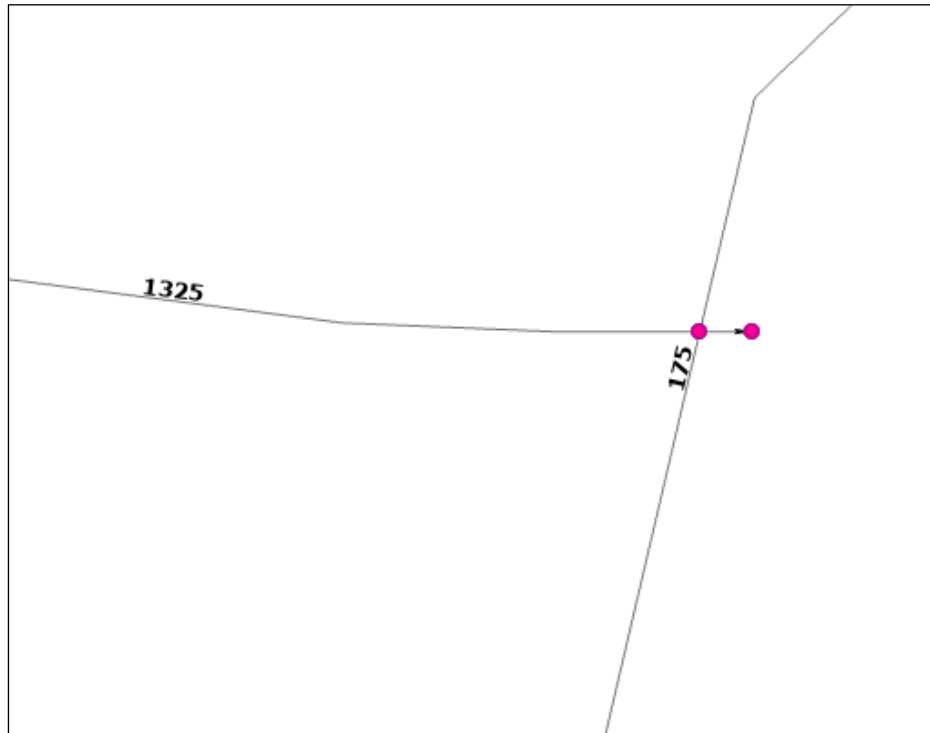
The result should look like this.



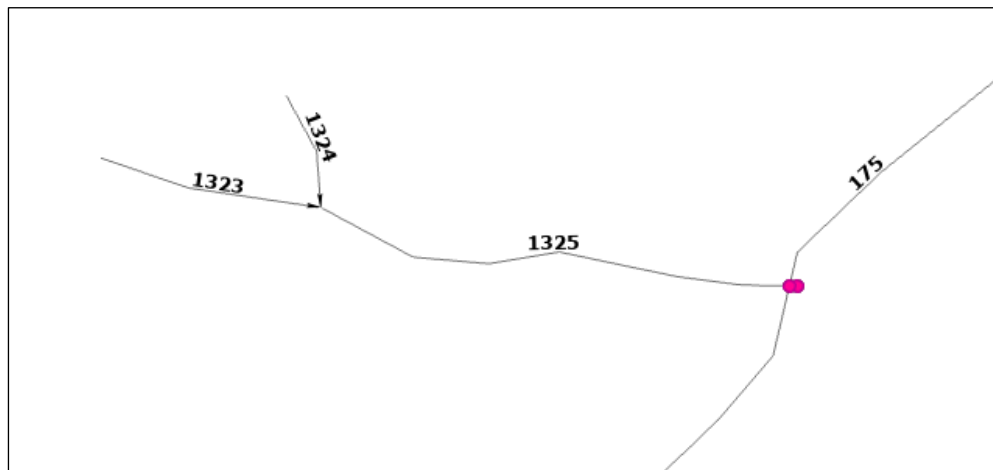
- e. Dangling Node 4. The final scenario illustrates a Dangling Node and an Intersection Without Node error that occurs when one reach crosses another without a valid

node intersection. Open the main.node_errors attribute table and manually select pointid = 1338 and pointid = <Null>.

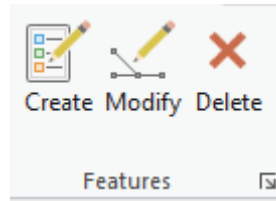
11	11	Point	1338	Outlet	Dangling Node
12	12	Point	<Null>	Confluence	Intersection Without Node



Note that two node errors are created. One where 1325 intersects 175, and another at the dangling node. Because we've split reaches and snapped endpoints previously, to correct this error we'll simply delete the small network causing the error.

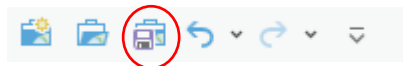


Click Edit to activate the editing ribbon. Manually select reaches 1323, 1324, and 1325. Click the Delete button in the features group.



Save edits.

Save the ArcGIS Pro project using the save button on the frame.

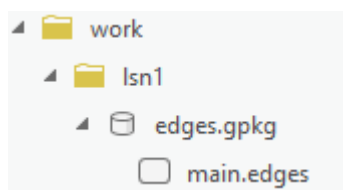


Remove all layers from the map window.

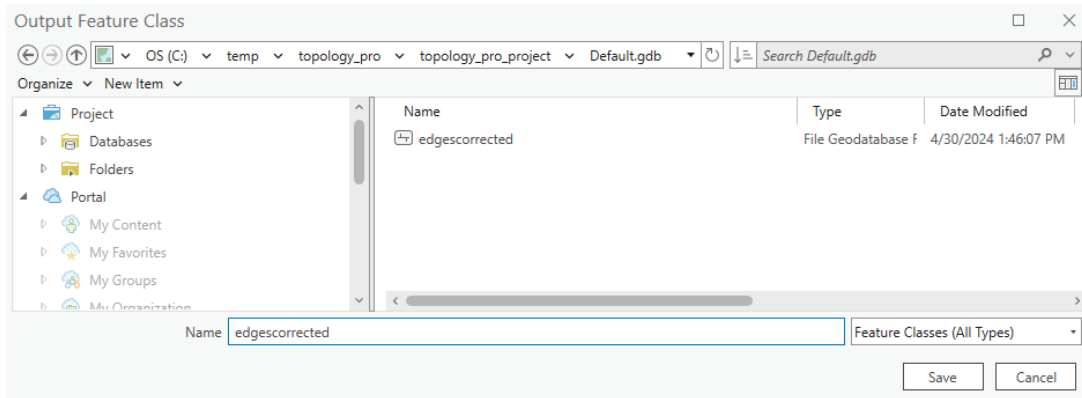
The editing exercise is complete, but now we will now create a new LSN to determine if all the errors were edited correctly.

9. Generate LSN2 and Check Node Errors

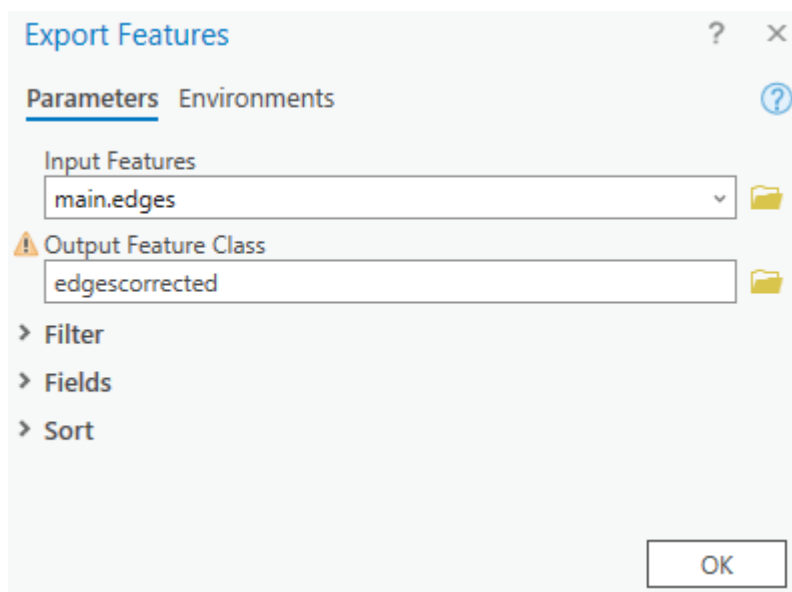
- a. Discussion: The R script generated an LSN by importing a shapefile called ARiver.shp and creating edges, nodes, node_errors, and .csv files that describe the topology. The edges, nodes, and node_errors were stored in a data model called a GeoPackage. GeoPackage is an open source, platform independent GIS model. ArcGIS Pro can read and write aspects of the GeoPackage format, but the feature topology may not be perfect after editing GeoPackages in Pro. To clean up potential topological problems it's a good idea to convert the edited GeoPackage edges to Geodatabase format, and then to shapefile for reintegrating into a new LSN.
- b. In Catalog, locate the feature class you just edited called main.edges and drag it into the Pro map viewer.



Right click main.edges in the Contents pane and select Data > Export Features. In the Export Features dialog box, choose a new Output Feature Class. Navigate to the default Geodatabase we created for the editing Project. It is inside the Geodatabase called **topology_pro_project**. Inside this project, double click on the GDB called Default.gdb. Name the output: edgescorrected.



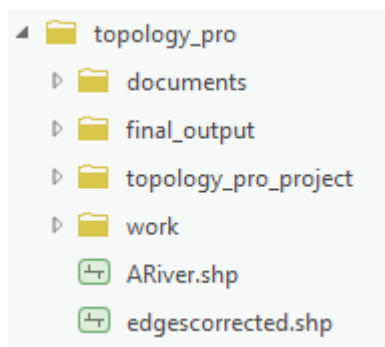
Click OK in the dialog box.



A new layer will be added to the map viewer Contents pane called edgescorrected. Now we will convert this feature class to a shapefile.

Open **Default.gdb** and right click **edgescorrected**. Select **Export > Feature class to Shapefile**. In this case, set the output location to **C:\temp\topology_pro**. **edgescorrected.shp** will be copied to this folder.

The shapefile should be here in Catalog.



- c. Open CheckTopologyPro.R in RStudio or any other IDE for R. Load the relevant R packages and set your working directory, which may be different to the one below. Run the R code step by step as described below if you quit R or R Studio in the midst of this tutorial.

```
## Load SSNbler and other useful packages
library(SSNbler)
library(sf)
## Load two more libraries for parallel processing
library(tidyverse)
library(parallel)

## Set number of cores to use for parallel processing
detectCores() # How many cores do I have?
num.cores <- 10 ## Don't all the cores. Leave a few to do other things.

## Set working directory
setwd("C:/temp/topology_pro")
```

- d. Import the edited shapefile and build LSN2.

```
## Import the streams dataset as an sf object
river_net<- st_read("ARiver.shp")

## Set output name for LSN
lsn_path1<- "c:/temp/topology_pro/work/lsn1"

## Cast the streamline shapefile as linestring format
river_net2 = st_cast(river_net,"LINESTRING")

## Build the initial LSN and check the topology
edges<- lines_to_lsn(
  streams = river_net2,
  lsn_path = lsn_path1,
  snap_tolerance = 1,
  check_topology = TRUE,
  topo_tolerance = 20,
  overwrite = TRUE,
  verbose = TRUE,
  remove_ZM = TRUE,
  use_parallel = TRUE,
  no_cores = num.cores)
```

- e. List files that were generated in the LSN.

```
## Check output files. If node_errors.gpkg exists, then there are
## potential errors to check
list.files(lsn_path2)
```

```
[1] "edges.gpkg"          "nodereationships.csv" "nodes.gpkg"          "nodexy.csv"
[5] "relationships.csv"
```

If there is no file named `node_errors.gpkg`, then there are no topological errors in the `edges.gpkg` feature class. Congratulations! If there are node errors, it's OK and there's no need to go back and correct those.